TDMA Schedule Optimizer and EMANE Integration

This document covers the design process, the reasoning behind each decision and the final values for both parts of the project. Part 1 is the Python schedule optimizer. Part 2 is the EMANE integration that enforces that schedule in a packet emulation.

Every factual claim is labelled with one of two tags:

- **[run]**: observed on this machine in a command run during development or final verification, or in the saved output of the EMANE run in `results/`.
- **[from docs, not run]**: taken from the brief, EMANE documentation or reasoning, and not measured here.

Where a value is computed from a configuration number and not measured, the text says so.

1. Problem restated [from docs, not run]

Sixteen radios share one frequency. They take turns in fixed time slots inside a repeating frame. Two radios may use the same slot only when they are far enough apart that neither interferes with the other's receivers. The job is to assign a slot to every radio so that no receiver ever hears two transmitters at once, while using as few slots as possible so the frame is short and throughput is high. This restates the brief.

2. Part 1: the schedule optimizer

2.1 From radios to a graph

Radios are points in a 2D plane. The brief says two radios can talk when they are within 500 m. That gives the connectivity graph G: one node per radio, one edge per pair at most 500 m apart. This is a unit-disk graph. NetworkX holds G. **[from docs, not run]** for the modelling choice, which comes from the brief.

Distance is plain Euclidean distance. A pair at exactly 500 m is treated as connected, matching "within 500 meters". **[run]**: the test `test_exact_range_boundary_is_a_link` passes.

2.2 Why distance-2 colouring [from docs, not run]

This section is reasoning from the brief and the distance-2 colouring literature. It was not measured. Section 5 and the unit tests check its consequences.

A slot assignment is collision-free when no receiver hears two transmitters in the same slot. Two transmitters A and C collide at a receiver B when:

- A and B are neighbours (B can hear A), and
- C and B are neighbours (B can hear C), and
- A and C transmit in the same slot.

So A and C conflict when they share a neighbour, that is when they are at most 2 hops apart in G. Collecting every such pair gives the conflict graph G squared (G2): an edge between any two nodes that are 1 hop or 2 hops apart. A proper vertex colouring of G2, where adjacent nodes get different colours, is exactly a collision-free schedule. Each colour is one slot.

The two interference cases in the brief map directly onto it:

- Direct link (distance 1): A and B share an edge, so they are adjacent in G2 and get different slots.
- Hidden terminal (distance 2): A and C both reach B but not each other. They are 2 hops apart, so they are adjacent in G2 and get different slots.

Spatial reuse follows from the same construction. Nodes more than 2 hops apart are not adjacent in G2, so the colouring may give them the same colour, which means the same slot. **[run]**: in the demo, slot 0 carries 4 nodes (Section 2.4), and `test_spatial_reuse_beyond_two_hops` passes.

2.3 Why finding the minimum is hard, and the three-layer solver

The minimum number of slots is the chromatic number of G2, which is NP-hard to find in general. **[from docs, not run]**

The solver in `tdma_scheduler.py` has three layers:

1. **Upper bound by greedy colourings. [run]**: the code tries five NetworkX greedy strategies (`largest_first`, `smallest_last`, `DSATUR`, `independent_set` and `connected_sequential_bfs`) and then 300 randomized DSATUR restarts by default. It keeps the best colouring found. DSATUR colours next the node that has the most distinctly coloured neighbours.
2. **Lower bound by max clique. [run]**: the code calls `nx.max_weight_clique` on G2. Every node in a clique of G2 is adjacent to every other, so each needs its own slot, and the clique size is a floor on the frame length. When the heuristic result equals this floor the answer is proven optimal.
3. **Exact search. [run]** for the code path, **[from docs, not run]** for its use on the sample: `colour_with_k` tries to find a colouring with one slot fewer than the best, and proves optimality when none exists. On the 16-node sample the heuristic already met the lower bound, so by reading `optimise` the exact search was not needed there. It was not separately traced. The brute-force test (`test_matches_brute_force_on_tiny_graphs`) compares the solver with exhaustive search on 25 random 6-node graphs only.

The report prints a lower bound, an upper bound and, when they meet, the words “Proven optimal”. It does not claim a general optimum. **[run]**

2.4 Part 1 result on the sample topology [run]

Running `python tdma_scheduler.py --demo` on the demo grid:

- 16 nodes, 42 links (1-hop edges), radio range 500 m.
- Frame length: **9 slots**.
- Lower bound (max clique): **9 slots**, so **Proven optimal: yes**.
- Spatial reuse: slot 0 carries 4 nodes, several slots carry 2, reuse factor about 1.78 nodes per slot.
- Solve time about 72 ms.

The matrix and the node-to-slot mapping are printed as an ASCII Slot x Node boolean matrix, as the brief asks. The brief shows a sample report with a 5-slot frame, but the coordinates behind that sample are not given. The frame length depends on the coordinates, so 5 and 9 cannot be compared. Nine is the proven optimum for the supplied `sample_nodes.json` coordinates only.

2.5 Tests [run]

`test_tdma_scheduler.py` has 16 tests and all pass (`python -m unittest -v test_tdma_scheduler`). The names below match the file.

`ScheduleCorrectness` (11 tests):

- `test_demo_grid`: the demo topology.
- `test_random_topologies_many_densities`: random 16-node layouts at five area sizes, each checked conflict-free.
- `test_matches_brute_force_on_tiny_graphs`: frame length equals the exhaustive minimum on 6-node graphs.
- `test_two_hop_hidden_terminal`: the distance-2 case from the brief.
- `test_spatial_reuse_beyond_two_hops`: far nodes share a slot.
- `test_exact_range_boundary_is_a_link`: a pair at exactly 500 m is a link.
- `test_duplicate_coordinates_are_adjacent`: nodes at the same point conflict.
- `test_fully_disconnected_uses_one_slot`: no links gives one slot.
- `test_fully_connected_needs_one_slot_per_node`: a full clique gives one slot per node.
- `test_deterministic_for_same_seed`: the same seed gives the same schedule.
- `test_verifier_catches_a_bad_schedule`: the checker rejects a bad schedule.

`InputHandling` (3 tests): `test_rejects_bad_json`, `test_rejects_wrong_shapes` and `test_natural_sort_order`.

`CommandLine` (2 tests): `test_json_argument` (the CLI accepts a JSON string and reports “Schedule verified conflict-free”) and `test_bad_input_exits_2_without_traceback`.

3. Part 2: EMANE integration

3.1 Goal [from docs, not run]

Take the Part 1 matrix and make a real EMANE emulation deliver or drop packets according to it. EMANE’s TDMA event scheduler radio model accepts a schedule as an event. The bridge converts the matrix into that event and builds the topology so the pathloss matches Part 1’s unit-disk assumption. Section 5 reports what the emulation did.

3.2 The bridge `emane_bridge.py`

It does three things. **[run]**: all three were exercised in the final verification, which regenerated the outputs and got files identical to the committed ones.

1. Loads the Part 1 output (`schedule.json`) or runs Part 1 from coordinates.
2. Writes the TDMA schedule XML for three variants (see 3.5).
3. Writes `pathloss.eel`, the per-pair pathloss that recreates the 500 m unit-disk topology inside EMANE.

Node names must be `Node_NN`. The integer NN becomes the EMANE NEM id, so the mapping is stable and readable. **[run]**: `test_nem_ids` passes.

3.3 Final radio and schedule values, with reasons

All of these live in one block at the top of `emane_bridge.py`. Labels in the last column say whether the value is only a configuration number or was observed.

Value	Setting	Reason	Label
Slot duration	1000 microseconds (1 ms)	The brief asks for 1 ms slots. EMANE wants microseconds.	**[from docs, not run]** for the unit. The value is in the XML that EMANE accepted, **[run]**.
Slot overhead	10 microseconds	EMANE needs overhead at least the maximum propagation delay. 500 m is 1.67 microseconds of flight, so 10 leaves margin for software timer jitter. Cost is 1 percent of a slot.	**[from docs, not run]**
Frames per multiframe	1	Part 1 produces one recurring frame.	**[run]**: the XML validates and EMANE accepted it.
Frequency	2.4 GHz	The 2.4 GHz ISM band, the band EMANE guide examples use. With precomputed pathloss the value only matters for matching transmit and receive frequency, and every NEM shares it.	**[from docs, not run]**. The PHY log showed `frequencyofinterest = 2400000000 Hz` after the fix, **[run]**.
Bandwidth	10 MHz	Sets the noise floor: -174 dBm/Hz + 10 log10(10e6) + 4 dB noise figure = -100 dBm.	The PHY log showed `bandwidth = 10000000 Hz`, **[run]**. The noise floor is computed, not measured.
Tx power	20 dBm (100 mW) in the schedule XML	A typical tactical handheld level. **Applied power is unresolved.** The PHY log showed `txpower = 0.00 dBm` in the same run. See Section 3.4.	The XML value is **[run]**. The power applied on the air was not established.
Data rate	9 Mbps	Picked from the TDMA PCR curves in `tdma_pcr.xml`. At 9 Mbps the probability of reception is 99.7 percent at 5 dB SINR, 100 percent at 6 dB and 0.3 percent at 0 dB. A clean link needs about 5 dB or more to succeed, and one equal-power interferer pulls SINR to about 0 dB, where reception almost always fails. That makes collisions visible.	PCR values read from the file, **[run]**. The SINR levels are computed.
Service class	0	Best effort queue, matches the default DSCP of the test traffic.	**[from docs, not run]**
In-range pathloss	94.0 dB	Free-space loss at 500 m and 2.4 GHz. Applied to every in-range pair so a link is either full quality or absent.	EEL contents checked against geometry, **[run]**.
Out-of-range pathloss	200.0 dB	Large enough that received power is far below the noise floor, so a far pair neither communicates nor adds interference.	EEL contents checked, **[run]**. Zero delivery measured on 156 out-of-range pairs, **[run]**.

3.4 Range enforcement and transmit power

How the 500 m limit is enforced. [run] for the EEL contents, **[from docs, not run]** for the model reasoning.

- The limit is enforced by the 200 dB pathloss value that `pathloss.eel` assigns to every out-of-range pair. No receiver sensitivity setting is configured anywhere in `emane_bridge.py` or `emulate.py`, so sensitivity plays no part.
- The cliff comes from this thresholding, not from free-space physics. True free-space loss at 600 m is 95.62 dB. With the same transmit power that would be only about 1.6 dB worse than at 500 m, a perfectly decodable signal. The 200 dB value is a deliberate modelling choice so that Part 1's unit-disk assumption holds exactly and a far transmitter cannot capture a receiver and hide a collision.
- All 240 ordered pairs in `pathloss.eel` match the 94 dB or 200 dB value implied by their distance in `sample_nodes.json`, with zero mismatches. **[run]**
- The 94 dB value at 499 m is the EEL builder's output for a test pair at 499 m (**[run]**). No 499 m pair exists in the sample, so no pair at that distance was run live. The nearest out-of-range pair run live was 600 m, and it delivered nothing (**[run]**).

Transmit power is unresolved. [run] for the logs, computed for the SNR figures.

- The schedule XML sets `power="20.0"` (dBm).
- The NEM PHY log in the same run showed `txpower = 0.00 dBm`. A debug trace taken during diagnosis also showed `power: 0.000000` on a transmit control message. Whether 20 dBm or 0 dBm was applied on the air was not resolved, and no run was done to settle it.

Computed SNR with the -100 dBm noise floor. These are computed from configuration values and pathloss, not measured:

Case	Tx 20 dBm (XML value)	Tx 0 dBm (PHY log value)
In range, 94 dB	-74 dBm received, SNR 26 dB	-94 dBm received, SNR 6 dB
Out of range, 200 dB	-180 dBm received, SNR -80 dB	-200 dBm received, SNR -100 dB

At 6 dB SNR the 9 Mbps PCR curve gives 100 percent reception, so an in-range clean link succeeds in both cases. One equal-power interferer pulls SINR to about 0 dB in both cases, where the curve gives 0.3 percent. The out-of-range pairs are far below the noise floor in both cases. The delivery results in Section 5 therefore hold whichever power was applied. Only the absolute received-power numbers are in doubt.

3.5 The three schedule variants

The roles below are the design intent. **[from docs, not run]** for the expectations. Section 5 reports what was measured.

- **part1**: the real Part 1 schedule. Expected result is high delivery and no collisions, because the schedule is collision-free by construction.
- **distance1**: a negative control that only separates 1-hop neighbours and ignores the 2-hop constraint. It should reintroduce hidden-terminal collisions and lower delivery for affected pairs.
- **allslot0**: a negative control that puts every node in slot 0. Everyone talks at once, so receivers should hear nothing, because a node that is transmitting cannot receive in the same slot.

The two controls exist so a successful part1 run can be compared against schedules that are expected to fail.

3.6 Schedule XML validates and round-trips [run]

- All three schedule XML files validate against EMANE's `tdmaschedule.xsd` (via lxml), with zero schema errors.
- `schedule_part1.xml` parses back to exactly the Part 1 matrix. The conversion loses nothing.

3.7 The emulation `emane/emulate.py`

What it does [from docs, not run] for the design, [run] for the saved results. The results in Section 5 come from a saved earlier run (20 s, 500 packets per second, 100 byte payload, per radio). They were not re-run during the final verification. The final verification re-read the saved `results/*/raw.json` files and confirmed the Docker image still contains EMANE 1.5.3.

One Linux network namespace per radio (`nem1` to `nem16`), each running its own `emane` process with the TDMA radio model, which is how separate radio boxes are modelled. A bridge `br0` carries the over-the-air multicast and the event multicast. `emaneeventservice` replays `pathloss.eel`. `emaneevent-tdmaschedule` publishes the chosen schedule XML. Then every radio broadcasts UDP for the run duration and counts what it hears from each neighbour, while EMANE's own accept and drop

tables are dumped.

The EMANE config XML files (platform, NEM, MAC, PHY, transport, event service) are generated at runtime by `emulate.py` so every value can be traced to the constant that produced it. The MAC points at `emane-tdma-pcr.xml` with 1 ms slots.

4. Tested vs not tested

Tested and verified on this machine [run]:

- Part 1: 16 unit tests pass, run again in final verification.
- Part 1 demo: 9-slot frame, proven optimal against the max-clique lower bound.
- Bridge: 5 unit tests pass. The full suite is 21 passed.
- All three schedule XML files validate against `tdmaschedule.xsd`.
- `schedule_part1.xml` round-trips to the Part 1 matrix.
- `pathloss.eel` matches the 500 m unit-disk topology for all 240 pairs.
- The EMANE live run, from a saved earlier run: the Docker image built, all 16 radios installed their schedules (`scheduleAcceptFull` was 1 on all 16 in all three variants) and all three variants ran. See Section 5 for numbers.

Computed or reasoned, not measured [from docs, not run]:

- Received power and SNR figures in Section 3.4 (computed from configuration values).
- The 94 dB value at 499 m (the EEL builder output was checked, but no 499 m pair was emulated).
- The reasoning for distance-2 colouring and the NP-hardness statement.

Not tested [from docs, not run]:

- The transmit power actually applied on the air (20 dBm in the XML against 0 dBm in the PHY log).
- A live run at 499 m. The nearest out-of-range pair run live was 600 m.
- A fresh EMANE run during final verification. The results are from a saved earlier run.
- The exact search on the sample topology. It was not needed there.
- Node mobility. The topology is static.
- Any topology other than the 16-node sample in the live run.
- Partial-quality links or a distance-based pathloss model. The model is a hard unit disk by choice (Section 3.4).
- Runs longer than 20 seconds or at other traffic rates.

The one measured zero in a collision-related counter is part1's SINR collision-drop counter, which read 0 bytes across all radios in the saved results (Section 5.2). Everywhere else the claim that part1 is collision-free rests on the colouring of G2 being proper, which is a property of the construction and was checked by the conflict-free verifier in the Part 1 tests, not by a packet measurement.

5. Live EMANE run status [run]

The EMANE Docker image (EMANE v1.5.3) was built, and all three variants ran on this machine for 20 seconds each, 500 broadcast datagrams per second per radio, 100 byte payload. The numbers below come from the saved output in `results/<variant>/raw.json`. They were not re-run in the final verification.

5.1 Per-variant results [run]

Variant	Slots	Schedule installed on all 16 NEMs	scheduler.scheduleAcceptFull	Clean-link PDR (in-range, slot-separated)	Collision or half-duplex PDR	Schedule agreement
part1	9	yes, 0 mismatches	1 on all 16	99.5 percent (84 pairs, min 98.8)	no such pairs exist	240 of 240 pairs
distance1	4	yes, 0 mismatches	1 on all 16	98.4 percent (20 pairs)	0.1 percent (64 collision pairs)	240 of 240 pairs
allslot0	9	yes, 0 mismatches	1 on all 16	no deliverable pairs exist	0.0 percent (84 half-duplex pairs)	240 of 240 pairs

"Schedule agreement" counts ordered pairs where EMANE's measured delivery matched the schedule prediction (deliver

and PDR of at least 0.9, or blocked and PDR of at most 0.1). All 240 ordered pairs agreed in every variant.

5.2 What each variant showed [run]

- **part1**: the real schedule. All 84 in-range slot-separated pairs delivered at 99.5 percent mean. The 156 out-of-range pairs delivered nothing, so the 500 m range held for the nearest of those, which is 600 m. The SINR drop counter, summed over all radios by `emane/analyze.py`, read 0 bytes. “0 measured collisions” in this repo means that counter reading in the saved results and nothing broader.
- **distance1**: 1-hop separation only, no 2-hop constraint. Clean-link delivery for the 20 pairs that stay clean was 98.4 percent, close to part1’s 99.5 percent, so clean-link delivery alone does not tell the variants apart. The main evidence of hidden-terminal collisions is the SINR drop counter, which showed 39,182,660 bytes dropped by the SINR check. The 64 pairs that the 2-hop constraint would have protected delivered 0.1 percent, which supports the same reading. This is the hidden-terminal problem made visible, and it is why the distance-2 colouring exists.
- **allslot0**: every node in slot 0. Every in-range pair is half duplex, since the receiver is also transmitting in the same slot, so delivery was 0.0 percent across all 84 pairs. The SINR counter read 0 because nothing reached the SINR check, so this variant shows half-duplex blocking and not interference.

The per-NEM `ScheduleInfoTable` showed each radio installed exactly its planned transmit slots, and `scheduler.scheduleAcceptFull` was 1 on all 16 radios in every variant. Raw per-radio counts and the accept, drop and slot-status tables are in `results/<variant>/raw.json`. `emane/analyze.py` prints the per-pair comparison.

6. Issues encountered

6.1 EMANE Docker build failed on missing build-essential [run]

The first image build stopped at `make deb` with:

```
dpkg-checkbuilddeps: error: Unmet build dependencies: build-essential:native
make: *** [Makefile:906: deb] Error 3
```

Cause: `make deb` runs `dpkg-buildpackage`, which calls `dpkg-checkbuilddeps`. That check looks for the `build-essential` meta-package by name. The Dockerfile had installed `gcc`, `g++` and `make` individually, which compiles code fine but does not satisfy the `build-essential` dependency. EMANE’s own `debian/control` at the v1.5.3 tag was checked to confirm the real build dependencies, then `build-essential`, `autotools-dev` and `fakeroot` were added by their exact declared names.

6.2 tee hid the real build exit code [run]

The first build command piped Docker through `tee`:

```
docker build ... 2>&1 | tee build.log
```

The pipeline reported exit 0 even though Docker had failed, because the exit code of a pipeline is the exit code of its last command, which was `tee`. The failure was only visible by reading the log, not from the exit status. Fix: either set `set -o pipefail` so the pipeline reports the first failure, or write the log directly with `docker build ... > build.log 2>&1` and read `$?`. The rebuild used the direct-log form and returned exit 0.

6.3 pathloss.eel written with CRLF aborted the event service [run]

The first live run delivered zero packets. Debug logs showed over-the-air frames arriving but every one dropped with “propagation model missing information”, and the event service log showed:

```
ABORT LoaderPathloss loader: Parameter conversion error.
200.0 invalid character in numeric: '' (pathloss.eel:1)
```

Cause: `pathloss.eel` was generated on Windows, so Python’s text mode turned each line ending into CRLF. EMANE’s EEL pathloss loader reads the trailing carriage return as part of the last number on the line, fails to parse it and aborts. With no event service, no pathloss reached the radios, and the precomputed propagation model dropped every frame. Fix: write the file with explicit Unix line endings, and add a `.gitattributes` entry so it stays LF on checkout.

6.4 Receiver frequency mismatch, suspected cause [run] for the logs

The PHY log showed `frequencyofinterest = 2347000000 Hz` (the EMANE default) while the schedule transmits at 2400 MHz. A

receiver tuned to a different frequency than the sender is a plausible reason for frames to be ignored, so `frequencyofinterest` was set to the schedule frequency in `emane/emulate.py`. This was a suspected cause and it was not isolated. The frequency setting was changed first, a rerun still delivered nothing, and the debug trace then exposed the CRLF abort from Section 6.3. No run tested either fix alone, so it is not known whether the frequency change was needed.

6.5 Open issue: applied transmit power [run] for the logs

See Section 3.4. The schedule XML sets 20 dBm and the PHY log showed 0 dBm. The issue was found in final verification and was not investigated further. The delivery results do not depend on which value applied, but the absolute received-power figures do.